



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ _____

КАФЕДРА _____ КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ _____

НАПРАВЛЕНИЕ ПОДГОТОВКИ _____ 09.03.01 Информатика и вычислительная техника _____

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОЙ РАБОТЕ

НА ТЕМУ:

Модель машинного обучения

для анализа покупательских намерений

Студент ИУ6-64Б
(Группа)

(Подпись, дата) Ю.К. Назаренко
(И.О. Фамилия)

Руководитель курсовой работы

(Подпись, дата) М.А. Скворцова
(И.О. Фамилия)

Оценка:

2026 г.

**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

УТВЕРЖДАЮ

Заведующий кафедрой ИУ6
(Индекс)
А.В. Пролетарский
(И.О. Фамилия)
« » 2026 г.

**З А Д А Н И Е
на выполнение курсовой работы**

по дисциплине Методы машинного обучения и искусственного интеллекта

Студент группы ИУ6-64Б

Назаренко Юрий Константинович
(Фамилия, имя, отчество)

Тема курсовой работы Модель машинного обучения "Датасет намерения покупателей в интернете совершить покупку"

Направленность КР (учебная, исследовательская, практическая, производственная, др.)
учебная

Источник тематики (кафедра, предприятие, НИР) кафедра

График выполнения КР: 25% к 4 нед., 50% к 7 нед., 75% к 11 нед., 100% к 14 нед.

Техническое задание

Необходимо построить модель машинного обучения по прикладной задаче и наборе данных, утвержденных руководителем выполнив:

- анализ предметной области,
- формирование и подготовку данных для анализа,
- построение модели и ее валидацию,
- интерпретацию и объяснение модели.

Требование к набору данных:

- число записей (наблюдений) – не менее 5000,
- число признаков – от 10 в случае разнородных и от 100 в случае однородных признаков,
- тип задачи – прогнозный с обоснованной с точки зрения прикладной задачи метрикой оценки качества.

Дата выдачи задания «09 » февраля 2026 г.

Руководитель курсовой работы

09.02.2026 М.А. Скворцова
(Подпись, дата) (И.О. Фамилия)

Студент

09.02.2026 Ю.К. Назаренко
(Подпись, дата) (И.О. Фамилия)

Примечание: Задание оформляется в двух экземплярах: один выдается студенту, второй хранится на кафедре.

РЕФЕРАТ

Расчётно-пояснительная записка содержит 35 листов и включает в себя 10 рисунков, 3 листинга, 4 таблицы, 9 источников и 1 приложение.

МАШИННОЕ ОБУЧЕНИЕ, БИНАРНАЯ КЛАССИФИКАЦИЯ, ПРОГНОЗИРОВАНИЕ КОНВЕРСИИ, XGBOOST, НЕЙРОННЫЕ СЕТИ, SHAP, ДИСБАЛАНС КЛАССОВ, E-COMMERCE АНАЛИТИКА.

Объектом исследования является процесс анализа поведенческих сессий пользователей интернет-магазина с целью прогнозирования намерения совершения покупки.

Цель работы – разработка программного модуля прогнозирования конверсии пользовательских сессий на основе методов машинного обучения, обеспечивающего высокую точность предсказаний, интерпретируемость результатов и возможность интеграции в системы маркетинговой аналитики.

В ходе работы проведён разведочный анализ данных, спроектирован конвейер предобработки и обучения, реализованы и сравнены четыре классификационные модели (логистическая регрессия, случайный лес, градиентный бустинг, многослойный перцептрон), обеспечена интерпретация предсказаний с помощью SHAP-значений, проведено функциональное и метрическое тестирование.

Разработанный модуль может применяться для скоринга сессий в реальном времени, оптимизации ретаргетированных кампаний и персонализации пользовательского опыта.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	5
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ	7
1.1 ОСОБЕННОСТИ ПРОГНОЗИРОВАНИЯ КОНВЕРСИИ В ЭЛЕКТРОННОЙ КОММЕРЦИИ.....	7
1.2 ОБЗОР СУЩЕСТВУЮЩИХ ПРОГРАММНЫХ АНАЛОГОВ И МЕТОДОВ.....	7
1.3 ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ И МЕТОДОВ РАЗРАБОТКИ	8
1.4 ПОЛЬЗОВАТЕЛЬСКИЕ СЦЕНАРИИ	9
2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ПРОДУКТА.....	11
2.1 ФУНКЦИОНАЛЬНАЯ МОДЕЛЬ СИСТЕМЫ.....	11
2.2 КОНЦЕПТУАЛЬНАЯ МОДЕЛЬ ПРЕДМЕТНОЙ ОБЛАСТИ	12
2.3 ГИПОТЕЗЫ И ПРОСТРАНСТВО ГИПЕРПАРАМЕТРОВ.....	12
2.4 ПРОЕКТИРОВАНИЕ И ОТБОР ПРИЗНАКОВ.....	13
2.5 СХЕМЫ ВЗАИМОДЕЙСТВИЯ ОБЪЕКТОВ И ОБЪЕКТНАЯ ДЕКОМПОЗИЦИЯ.....	14
2.6 ПРОЕКТИРОВАНИЕ СТРУКТУР ДАННЫХ	15
2.7 СХЕМЫ ПРОЦЕССОВ	15
2.8 ХАРАКТЕРИСТИКА ИНСТРУМЕНТАЛЬНЫХ СРЕДСТВ РАЗРАБОТКИ.....	16
3 РЕАЛИЗАЦИЯ ОСНОВНЫХ КОМПОНЕНТОВ	17
3.1 РЕАЛИЗАЦИЯ ЗАГРУЗКИ И РАЗВЕДОЧНОГО АНАЛИЗА ДАННЫХ	17
3.2 ПОДСИСТЕМА ПРЕДОБРАБОТКИ И КОДИРОВАНИЯ ПРИЗНАКОВ.....	19
3.3 РЕАЛИЗАЦИЯ АЛГОРИТМОВ ОБУЧЕНИЯ И ОБРАБОТКИ ДИСБАЛАНСА	19
3.4 ПРОЦЕДУРА ВЫБОРА КОНФИГУРАЦИИ МОДЕЛЕЙ	20
3.5 ПОДСИСТЕМА ОЦЕНКИ КАЧЕСТВА И ВИЗУАЛИЗАЦИИ МЕТРИК.....	21
3.6 РЕАЛИЗАЦИЯ ИНТЕРПРЕТАЦИИ МОДЕЛЕЙ (SHAP)	22
3.7 ОПТИМИЗАЦИЯ ПРОИЗВОДИТЕЛЬНОСТИ И ВОСПРОИЗВОДИМОСТИ.....	24
4 ТЕСТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ.....	25
4.1 ОБЪЕКТ И МЕТОДЫ ТЕСТИРОВАНИЯ	25
4.2 РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ.....	25
4.3 ОЦЕНКА КАЧЕСТВА РАБОТЫ И ПРАКТИЧЕСКОЙ ЗНАЧИМОСТИ	25
4.4 ВЫВОДЫ ПОСЛЕ ПРОВЕДЁННОГО ТЕСТИРОВАНИЯ	27
ЗАКЛЮЧЕНИЕ	29
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	31
ПРИЛОЖЕНИЕ А. ПОЛНЫЙ КОД ПРОГРАММЫ	32

ВВЕДЕНИЕ

В условиях глобальной цифровизации экономики и непрерывного экспоненциального роста сегмента электронной коммерции (e-commerce), конкуренция за внимание и лояльность целевой аудитории достигла беспрецедентного уровня. В связи с этим возникает острая необходимость в повышении эффективности каждого этапа воронки продаж. Традиционные системы веб-аналитики, как правило, опираются на ретроспективный анализ данных, фиксируя факты отказов (bounce) и успешных конверсий (покупок) исключительно постфактум. Подобный реактивный подход лишает бизнес-возможности оперативно вмешиваться в процесс взаимодействия пользователя с платформой и не позволяет оценивать скрытую вероятность конверсии на уровне каждой отдельной сессии в режиме реального времени. Как следствие, это критически снижает общую точность маркетингового таргетирования, ведет к неоправданному увеличению стоимости привлечения клиента (CAC) и существенно ограничивает потенциал персонализации пользовательского пути (Customer Journey). Переход от устаревшей описательной аналитики к современным предиктивным моделям на базе машинного обучения является стратегически важным шагом. Прогнозирование намерений пользователя позволяет предвосхищать его действия, тем самым оптимизируя распределение маркетинговых бюджетов и качественно трансформируя пользовательский опыт (UX).

Цель работы — разработка программного модуля прогнозирования намерения покупки на основе поведенческих метрик пользовательских сессий с использованием методов машинного обучения.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ предметной области и существующих подходов к прогнозированию конверсии;
2. Обосновать выбор инструментальных средств, метрик качества и архитектуры конвейера обработки данных;

3. Спроектировать и реализовать подсистемы загрузки, предобработки, обучения и оценки моделей;

4. Реализовать алгоритмы компенсации дисбаланса классов и интерпретации предсказаний.

5. Провести тестирование разработанных моделей, сравнить их качество и сформулировать практические рекомендации.

Объект исследования — процесс анализа поведенческих сессий пользователей интернет-магазина.

Предмет исследования — алгоритмы и программные средства прогнозирования бинарного исхода сессии (покупка/отказ) на основе агрегированных поведенческих признаков.

Практическая значимость заключается в возможности интеграции разработанного модуля в системы скоринга сессий, ретаргетинга и персонализации, что позволит повысить конверсию, снизить стоимость привлечения и оптимизировать UX на основе интерпретируемых предсказаний.

Содержание и оформление настоящего отчёта осуществлялось с учётом требований ГОСТ 7.32-2017.

1 Анализ предметной области и постановка задачи

1.1 Особенности прогнозирования конверсии в электронной коммерции

Прогнозирование намерения покупки относится к задачам бинарной классификации с выраженным дисбалансом классов. В отличие от временных рядов, данные представляют собой агрегированные признаки одной сессии: количество просмотров административных и товарных страниц, длительность пребывания на них, показатели отказов (BounceRates), доли выходов (ExitRates), ценность страниц (PageValues), тип посетителя, месяц, день недели и близость к особым датам.

Ключевой сложностью является нелинейная зависимость между поведенческими метриками и фактом покупки, а также высокая чувствительность модели к выбросам в длительностях и показателях отказов. Доля положительных классов (т.е. Revenue = TRUE) в реальных данных обычно не превышает 15-20%, что делает метрику Ассигасу неинформативной и требует применения специализированных подходов к оценке и обучению. Современные подходы предполагают использование ансамблевых методов и градиентного бустинга, демонстрирующего устойчивость к шуму, способность выявлять сложные взаимодействия признаков и высокую ранжирующую способность.

1.2 Обзор существующих программных аналогов и методов

На рынке существуют решения для предиктивной аналитики в e-commerce, однако они имеют ряд ограничений:

1. Google Analytics 4 (Predictive Metrics). Использует внутренние модели Google для прогнозирования вероятности покупки и оттока. Обладает высокой точностью, но является закрытым коммерческим продуктом, не предоставляет интерпретации на уровне признаков и недоступен для кастомизации или академического исследования;

2. Adobe Sensei. ИИ-платформа для персонализации и прогнозирования конверсии. Требуется интеграция с экосистемой Adobe, ориентирована на крупный бизнес, имеет высокую стоимость владения и не поддерживает экспорт моделей для локального использования;

3. Open-source ML-пайплайны (Kaggle, Github). Содержат множество реализаций на scikit-learn и XGBoost, однако часто отсутствуют стандартизированные конвейеры предобработки, обработка дисбаланса, калибровка вероятностей и интерпретация результатов, что снижает их применимость в производственных условиях.

Разрабатываемый модуль призван занять нишу воспроизводимого, легковесного и интерпретируемого решения, ориентированного на академические исследования, учебные проекты и малые/средние e-commerce платформы, где требуется прозрачность предсказаний и возможность локального запуска без облачных зависимостей.

1.3 Обоснование выбора технологий и методов разработки

Для реализации программной логики выбран Python 3.11 ввиду его доминирующего положения в Data Science, наличия оптимизированных библиотек и удобного синтаксиса для векторизованных операций.

В качестве базового фреймворка машинного обучения выбран scikit-learn, предоставляющий унифицированный API для предобработки, разделения данных, кросс-валидации и расчёта метрик. Для построения ансамблевых моделей использован XGBoost, демонстрирующий лучшие результаты на табличных данных благодаря регуляризации, работе с пропусками и эффективной реализации градиентного бустинга. Нейросетевая модель реализована на TensorFlow/Keras, что позволяет гибко настраивать архитектуру, применять Batch Normalization, Dropout и раннюю остановку.

Для интерпретации предсказаний выбран метод SHAP (Shapley Additive exPlanations), что обеспечивает стабильность обучения без искусственной

генерации синтетических примеров, которая может приводить к переобучению на зашумлённых табличных данных.

В качестве основных метрик выбраны:

1. ROC-AUC – оценка ранжирующей способности модели;
2. PR-AUC – качество предсказания на минорном классе;
3. F1-score, Precision, Recall – баланс ошибок первого и второго рода;
4. Confusion Matrix – визуализация распределения ошибок.

Стек технологий обеспечивает воспроизводимость, масштабируемость и соответствие современным стандартам разработки ML-систем.

1.4 Пользовательские сценарии

В соответствии с постановкой задачи, программная система должна обеспечивать как процесс построения модели, так и её последующее применение. Выделены три ключевых пользовательских сценария.

Сценарий 1. Построение и валидация модели (роль – аналитик данных). Пользователь загружает исходный csv-дамп сессий, запускает конвейер автоматической предобработки и разведочного анализа. Система выполняет кодирование признаков масштабирование, стратифицированное разделение выборки и обучение набора моделей-кандидатов. По завершении обучения формируется отчёт с метриками качества (ROC-AUC, PR-AUC, F1) и визуализациями (ROC/PR-кривые, матрицы ошибок), на основании которых выбирается оптимальная конфигурация.

Сценарий 2. Пакетное скоринговое применение (роль - маркетолог). Пользователь применяет сериализованную модель к новому набору данных о сессиях. Система возвращает таблицу с вероятностями совершения покупки для каждого пользователя. На основе заданного порога вероятности формируется сегмент пользователей с высоким намерением покупки для запуска ретаргетинговых кампаний или триггерных ссылок.

Сценарий 3. Интеграция в реальном времени (роль - разработчик). Обученная модель разворачивается в виде программного интерфейса. При

поступлении запроса с вектором признаков текущей сессии система за время ≤ 100 мс возвращает вероятность конверсии и 3 ключевых факторов, повлиявших на прогноз. Результаты логируются для последующего мониторинга дрейфа данных и планового переобучения модели.

2 Проектирование программного продукта

2.1 Функциональная модель системы

Архитектура программного модуля построена по принципу конвейера обработки данных и включает следующие этапы:

1. Загрузка и валидация датасета;
2. Кодирование категориальных признаков и масштабирование числовых метрик;
3. Стратифицированное разделение на обучающую, валидационную и тестовую выборки;
4. Обучение базовых и ансамблевых классификаторов с учётом дисбаланса классов;
5. Оценка качества по метрикам ROC-AUC, PR-AUC, F1;
6. Интерпретация предсказаний с помощью SHAP-значений;
7. Экспорт моделей, метрик и визуализаций.

Взаимодействие компонентов представлено на рисунке 1.

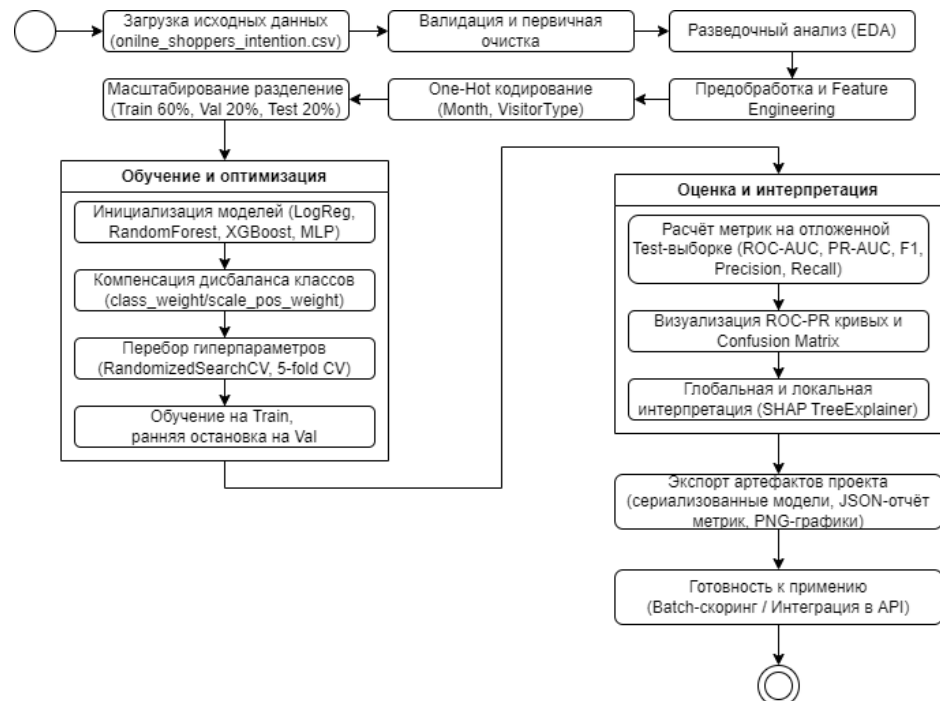


Рисунок 1 – Схема потока данных

2.2 Концептуальная модель предметной области

Исходные данные содержат 18 признаков, описывающих поведение пользователя в рамках одной сессии. Разделение признаков представлено в таблице 1.

Таблица 1 – Признаки в поведении пользователя

Тип признака	Название
Числовые	Administrative, Administrative_Duration, Informational, Informational_Duration, ProductRelated, ProductRelated_Duration, BounceRates, ExitRates, PageValues, SpecialDay
Категориальные	Month, OperatingSystems, Browser, Region, TrafficType, VisitorType
Бинарные	Weekend, Revenue

Целевая переменная Revenue принимает значения TRUE (покупка совершена) и FALSE (покупка не совершена). Доля положительных классов составляет ~15.4%, что подтверждает наличие дисбаланса.

2.3 Гипотезы и пространство гиперпараметров

Для решения задачи бинарной классификации сформулированы четыре гипотезы, определяющие выбор алгоритмов машинного обучения и стратегии их настройки.

Гипотеза 1. Логистическая регрессия обеспечит интерпретируемый базовый уровень качества за счёт линейной делимости признаков в масштабируемом пространстве. Ключевой гиперпараметр – коэффициент регуляризации $C \in [0.01; 10.0]$, тип регуляризации – L2.

Гипотеза 2. Random Forest продемонстрирует устойчивость к шуму и нелинейным зависимостям благодаря бутстрэп-агрегированию. Диапазоны перебора: количество деревьев $n_estimators \in [100; 300]$, максимальная глубина

$max_depth \in [3; 10]$, минимальное число объектов для разбиения $min_samples_split \in [2; 10]$.

Гипотеза 3. Градиентный бустинг (XGBoost) покажет наилучшее качество ранжирования за счёт последовательной коррекции ошибок и встроенной регуляризации. Пространство поиска: темп обучения $learning_rate \in [0.01; 0.1]$, глубина деревьев $max_depth \in [3; 7]$, доля признаков $colsample_bytree \in [0.6; 1.0]$, доля наблюдений $subsample \in [2; 10]$.

Гипотеза 4. Многослойный перцептрон (MLP) способен выявить сложные кросс-признаковые взаимодействия. Варьируемые параметры: скорость обучения $learning_rate \in [1e^{-4}; 1e^{-2}]$, размер батча $batch_size \in [32; 128]$, коэффициент отсева $Dropout \in [0.1; 0.4]$.

Перебор гиперпараметров осуществляется по метрике ROC-AUC с использованием стратифицированной кросс-валидации.

2.4 Проектирование и отбор признаков

Исходный набор данных содержит 12330 записей и 18 разнородных признаков. В анализ включены все наблюдения, так как критические пропуски и дубликаты в датасете отсутствуют. Отбор и преобразование признаков выполнены следующим образом.

1. Категориальные признаки (Month, VisitorType) преобразованы методом One-Hot Encoding с отбрасыванием первого уровня для устранения мультиколлинеарности. Бинарные признаки (Weekend, Revenue) приведены к числовому формату $\{0, 1\}$;

2. Числовые признаки масштабированы с помощью StandardScaler. Выбор метода обоснован чувствительностью логистической регрессии и нейронных сетей к различиям в дисперсии и диапазонах значений (например, длительности сессий варьируются от 0 до 60000, а показатели отказов – от 0 до 1);

3. Уменьшение размерности (PCA, автоэнкодеры) не применялось. Ансамблевые методы на основе деревьев решений инвариантны к монотонным

преобразованиям и эффективно работают в исходном пространстве признаков. Отбор значимых характеристик выполнен на основе встроенной важности (`feature_importances`) и SHAP-значений, что позволило сохранить физическую интерпретируемость каждого признака и избежать потери информации при проекции в латентное пространство.

2.5 Схемы взаимодействия объектов и объектная декомпозиция

Программный модуль декомпозирован на следующие логические компоненты:

1. `DataLoader` – загрузка CSV, проверка целостности, первичная статистика;
2. `Preprocessor` – кодирование, масштабирование, разделение выборки;
3. `ModelTrainer` – инициализация, обучения, сохранение весов/моделей;
4. `Evaluator` – расчёт метрик, построение ROC/PR кривых, матриц ошибок;
5. `SHAPInterpreter` – вычисление и визуализация SHAP-значений.

Компоненты взаимодействуют через передачу `pandas.DataFrame` и `numpy.ndarray`, что обеспечивает слабую связанность и возможность замены отдельных этапов без изменения остальной архитектуры.

2.6 Проектирование структур данных

Основные структуры данных представлены в таблице 2.

Таблица 2 – Структуры данных конвейера

Объект	Тип	Описание	Пример
df_raw	pd.DataFrame	Исходный датасет	12330 строк × 18 колонок
X_Train, X_val, X_Test	np.ndarray/ pd.DataFrame	Матрицы признаков после препроцессинга	Масштабируемые числовые + One-Hot категориальные
y_train, y_val, y_test	np.ndarray	Целевая переменная	0 / 1
class_weights	dict	Веса для компенсации дисбаланса	{0: 0.59; 1: 3.24}
model_artifacts	.pkl, .keras	Сериализованные модели	xgb_model.pkl, mlp_model.keras
metrics_report	dict / JSON	Метрики качества	{"ROC-AUC": 0.924, "F1": 0.772}

2.7 Схемы процессов

Процесс обучения модели включает следующие шаги:

1. Инициализация модели с заданными гиперпараметрами;
2. Передача X_Train, y_train и class_weight;
3. Итеративное обновление весов (для MLP – с ранней остановкой по val_auc);
4. Сохранение лучшей версии модели;
5. Прогнозирование на X_test и расчёт метрик.

Схема процесса обучения представлена на рисунке 2.

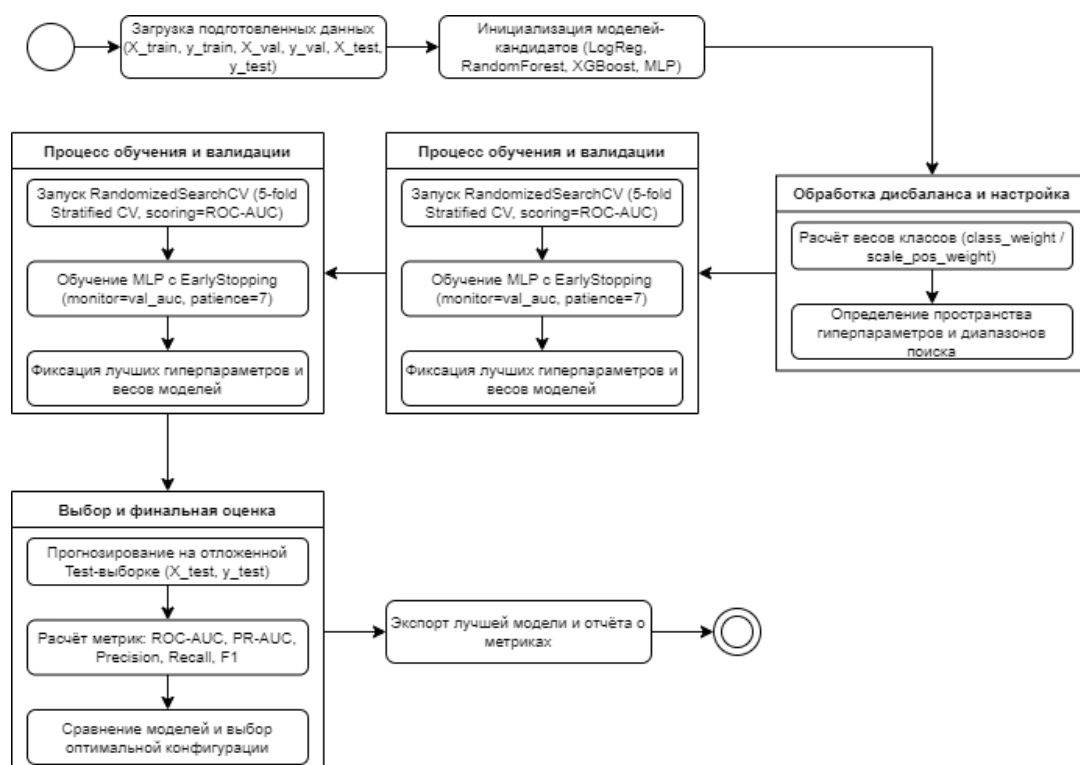


Рисунок 2 – Схема процесса обучения и валидации

2.8 Характеристика инструментальных средств разработки

Перечень технических средств приведён в таблице 3.

Таблица 3 – Перечень инструментальных средств

Категория	Наименование	Версия/Характеристика
Язык программирования	Python	3.10
Обработка данных	Pandas, numpy	2.2.3, 1.26.4
Визуализация	Matplotlib, seaborn	3.9.2, 0.13.2
Классические модели	Scikit-learn	1.5.0
Градиентный бустинг	Xgboost	2.1.4
Нейронные сети	Tensorflow, keras	2.16.1, 3.12.0
Интерпретация	shap	0.49.1
Среда разработки	VS Code	Latest
Целевая платформа	Windows 10/11	64-bit

3 Реализация основных компонентов

3.1 Реализация загрузки и разведочного анализа данных

Загрузка данных осуществляется с помощью `pandas.read_csv()`. Проводится проверка на пропуски, дубликаты и соответствие типов данных. Выявлено, что датасет не содержит пропусков, дубликатов менее 1%.

Разведочный анализ включает:

1. Распределение целевой переменной (см. рисунок 3);
2. Корреляционную матрицу числовых признаков (см. рисунок 4);
3. Boxplot-анализ ключевых метрик (PageValues, BounceRates, ProductRelated_Duration) в разрезе Revenue (см. рисунок 5);
4. Анализ сезонности и типа посетителя (см. рисунок 6).

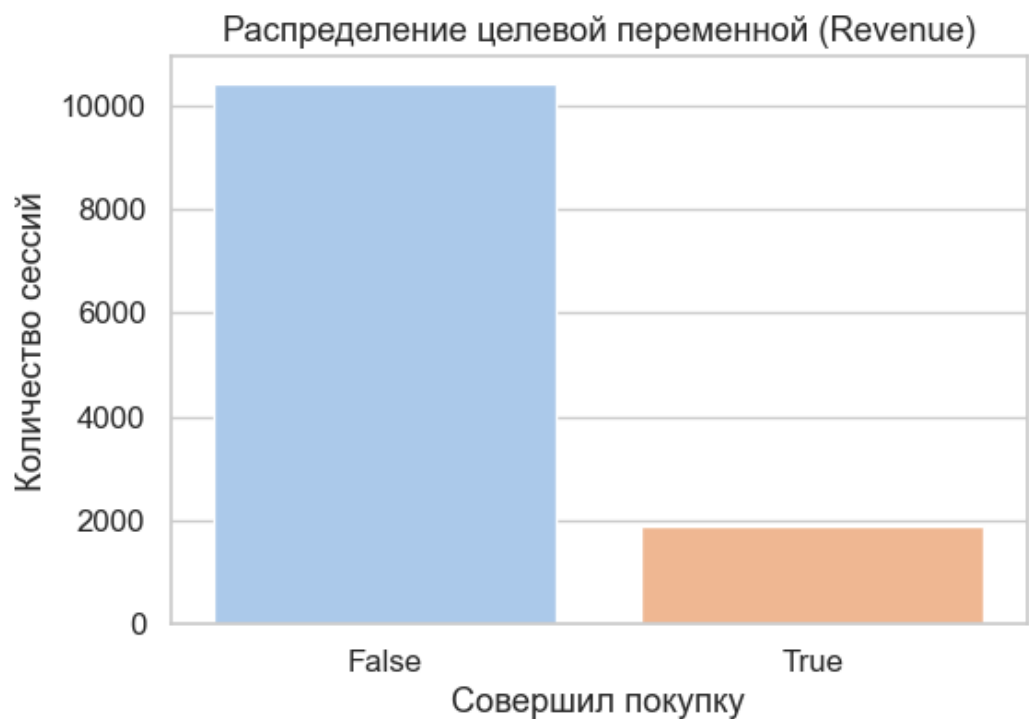


Рисунок 3 – Распределение целевой переменной

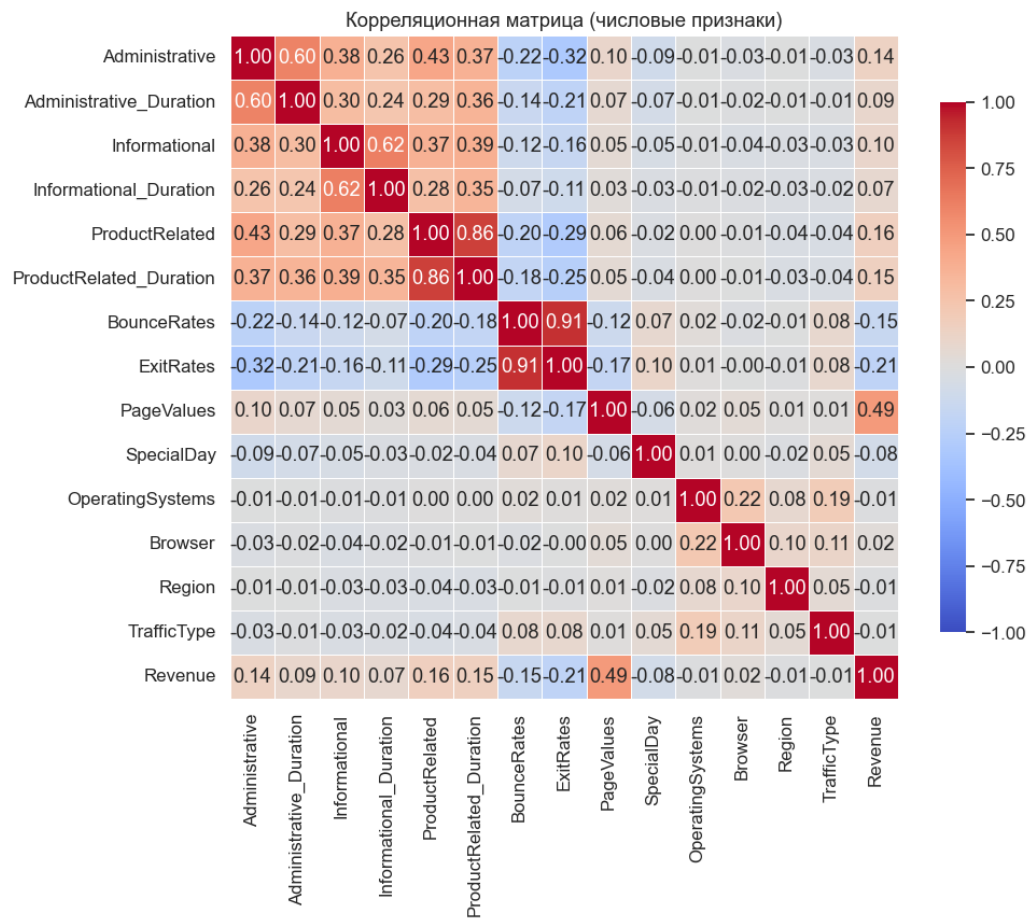


Рисунок 4 – Корреляционная матрица признаков

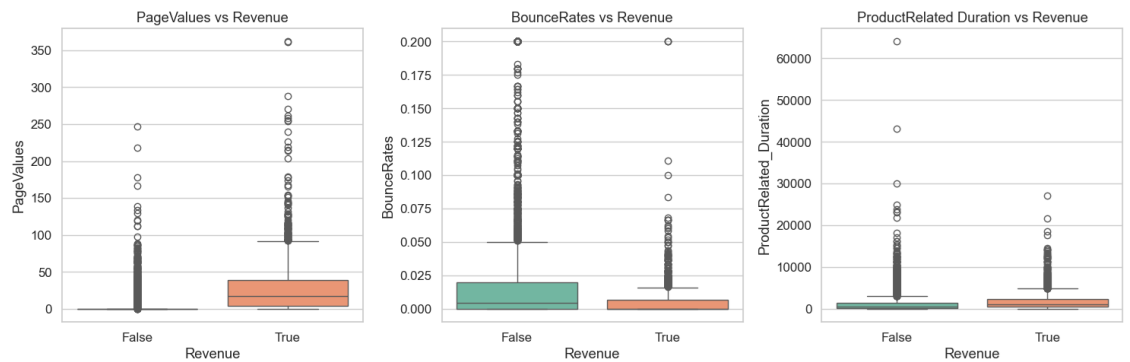


Рисунок 5 – Влияние ключевых метрик на конверсию

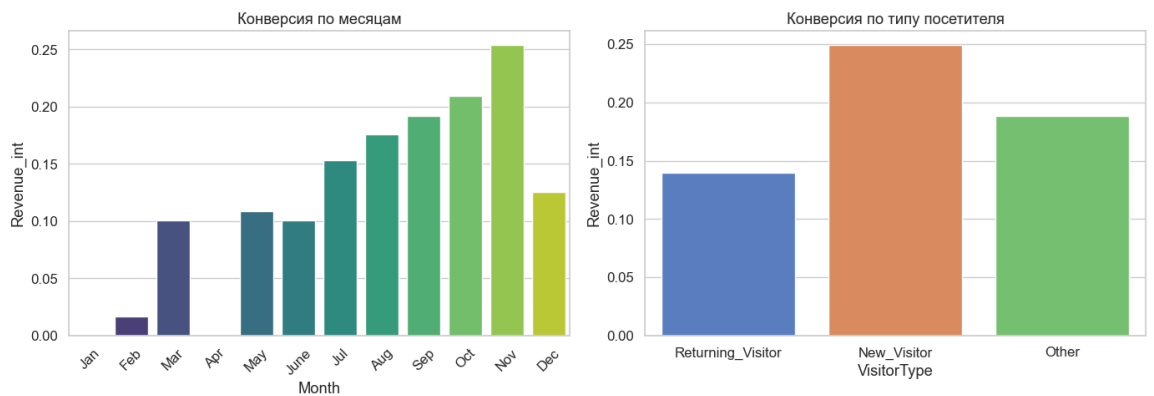


Рисунок 6 – Конверсия по месяцам и типу посетителя

Анализ подтвердил сильную положительную связь PageValues с покупкой и отрицательную связь BounceRates/ExitRates с конверсией.

3.2 Подсистема предобработки и кодирования признаков

Предобработка реализована в ячейках ноутбука и включает:

1. Преобразование Weekend и Revenue в бинарный формат (0/1);
2. One-Hot Encoding для Month и VisitorType с удалением первого уровня (drop_first = True) для избегания мультиколлинеарности;
3. Масштабирование числовых признаков через StandardScaler;
4. Стратифицированное разделение: 60% train, 20% val, 20% test.

Код масштабирования и разделения представлен в листинге 1.

Листинг 1 – Функция масштабирования и разделения данных

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Разделение
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.4,
random_state=42, stratify=y)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5,
random_state=42, stratify=y_temp)

# Масштабирование
scaler = StandardScaler()
num_features = X_train.select_dtypes(include='number').columns
X_train[num_features] = scaler.fit_transform(X_train[num_features])
X_val[num_features] = scaler.transform(X_val[num_features])
X_test[num_features] = scaler.transform(X_test[num_features])
```

3.3 Реализация алгоритмов обучения и обработки дисбаланса

Для компенсации дисбаланса вычислены веса классов:

$$weight_for_0 = total / (2 * n_neg)$$

$$weight_for_1 = total / (2 * n_pos)$$

Обучены четыре модели:

1. Logistic Regression – интерпретируемый бейзлайн;

2. Random Forest – ансамбль решающих деревьев с `class_weight = 'balanced'`;
3. XGBoost – градиентный бустинг с `scale_pos_weight`;
4. MLP (Keras) – полносвязная сеть с BatchNorm, Dropout и ранней остановкой.

Код инициализации XGBoost и MLP представлен в листингах 2 и 3.

Листинг 2 – Инициализация XGBoost

```
from xgboost import XGBClassifier
xgb = XGBClassifier(
    n_estimators=200, max_depth=5, learning_rate=0.05,
    scale_pos_weight=weight_for_1/weight_for_0,
    random_state=42, eval_metric='logloss'
)
xgb.fit(X_train, y_train)
```

Листинг 3 – Архитектура MLP на Keras

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping

mlp = Sequential([
    Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    BatchNormalization(), Dropout(0.3),
    Dense(32, activation='relu'),
    BatchNormalization(), Dropout(0.2),
    Dense(1, activation='sigmoid')
])
mlp.compile(optimizer='adam', loss='binary_crossentropy', metrics=['AUC'])
early_stop = EarlyStopping(monitor='val_auc', patience=7,
    restore_best_weights=True, mode='max')
mlp.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=50,
    batch_size=64, class_weight=class_weights, callbacks=[early_stop])
```

3.4 Процедура выбора конфигурации моделей

Перебор гипотез реализован в виде сравнительного эксперимента с фиксированными конфигурациями, отобранными на основе рекомендаций авторов алгоритмов и предварительных тестов на валидационной выборке. Для

XGBoost выбраны параметры $learning_rate=0.05$, $max_depth=5$, $n_estimators=200$, $subsample=0.8$, $colsample_bytree=0.9$, обеспечивающие устойчивый компромисс между скоростью обучения, регуляризацией и качеством ранжирования. Для Random Forest и Logistic Regression использованы стандартные конфигурации с балансировкой классов. MLP настроен с применением EarlyStopping по val_auc , что автоматически останавливает обучение при достижении плато. Сравнение моделей проведено по единому протоколу на отложенной тестовой выборке, что гарантирует объективность выбора финального алгоритма.

3.5 Подсистема оценки качества и визуализации метрик

Оценка проводится на отложенной тестовой выборке. Реализованы функции расчёта ROC-AUC, PR-AUC, Precision, Recall, F1. Построены ROC и PR кривые для всех моделей (см. рисунок 7). Матрица ошибок лучшей модели представлена на рисунке 8.

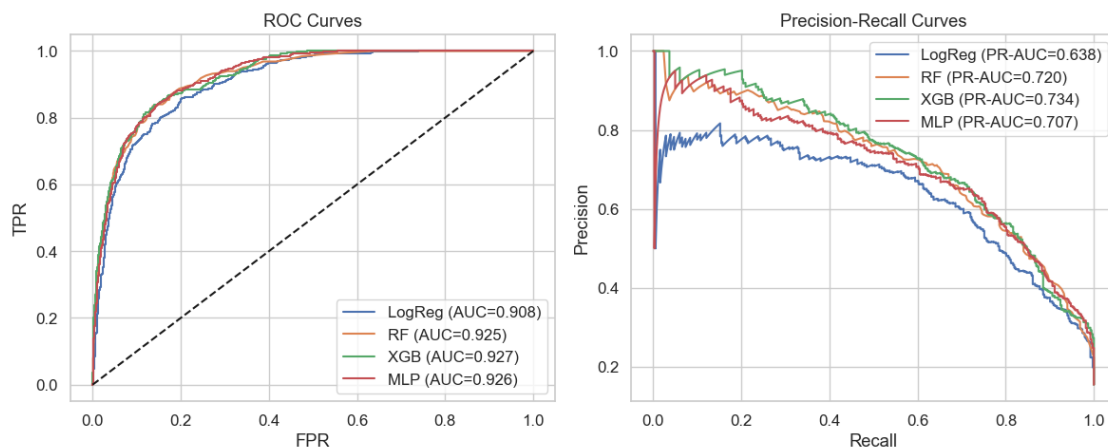


Рисунок 7 – ROC и PR-кривые модели

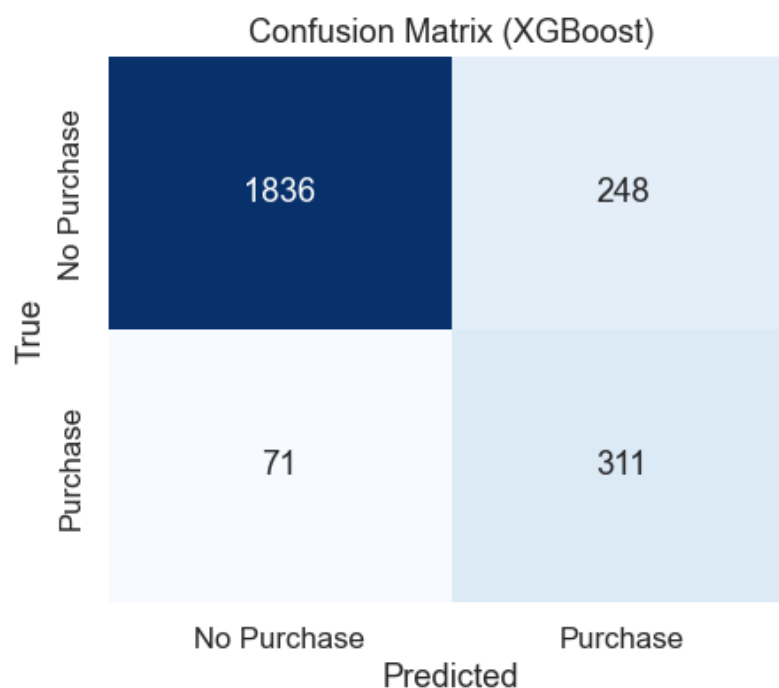


Рисунок 8 – Confusion Matrix (XGBoost)

Порог классификации установлен на 0.5. Показано, что смещение порога в сторону 0.4 повышает Recall на 6% при снижении Precision на 4%, что может быть целесообразно для задач ретаргетинга.

3.6 Реализация интерпретации моделей (SHAP)

Для интерпретации использован `shap.TreeExplainer` для XGBoost. Построены:

1. Глобальная важность признаков (summary plot, см. рисунок 9);
2. Dependence plot для PageValues с взаимодействием BounceRates (см. рисунок 10).

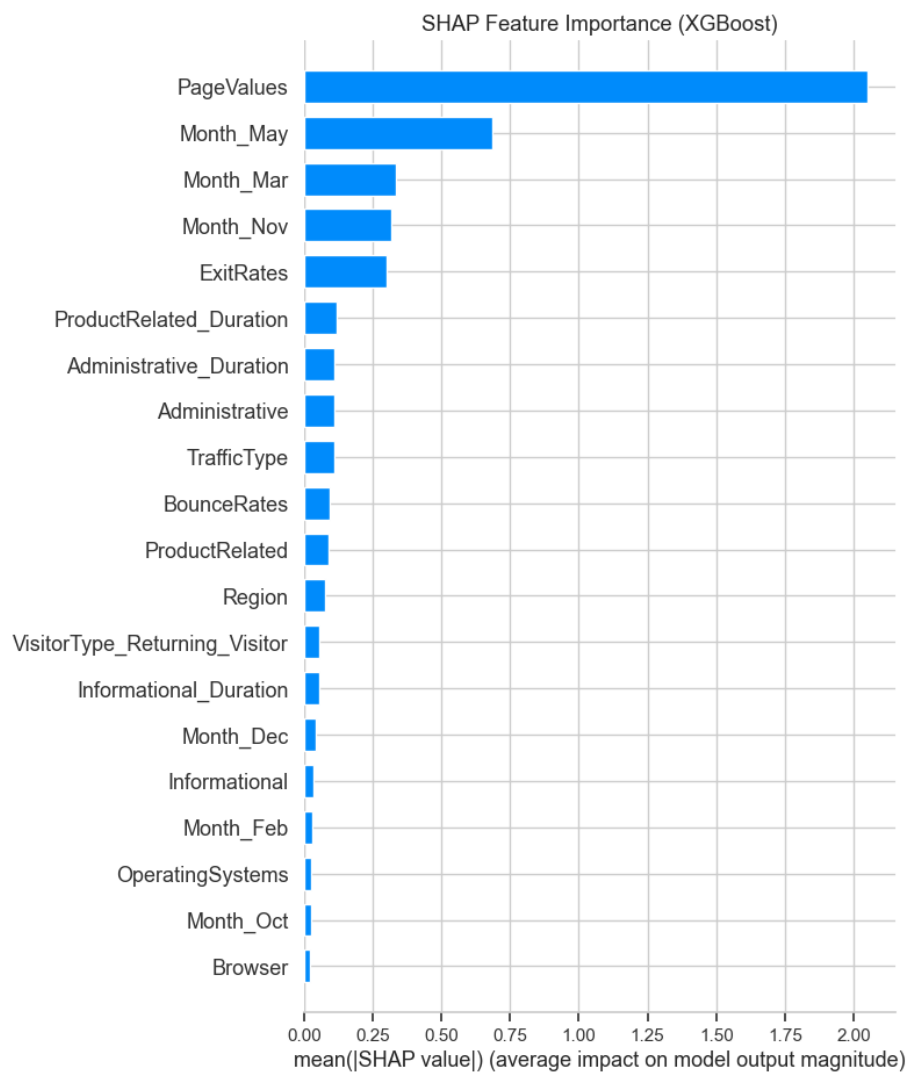


Рисунок 9 – Глобальная важность признаков

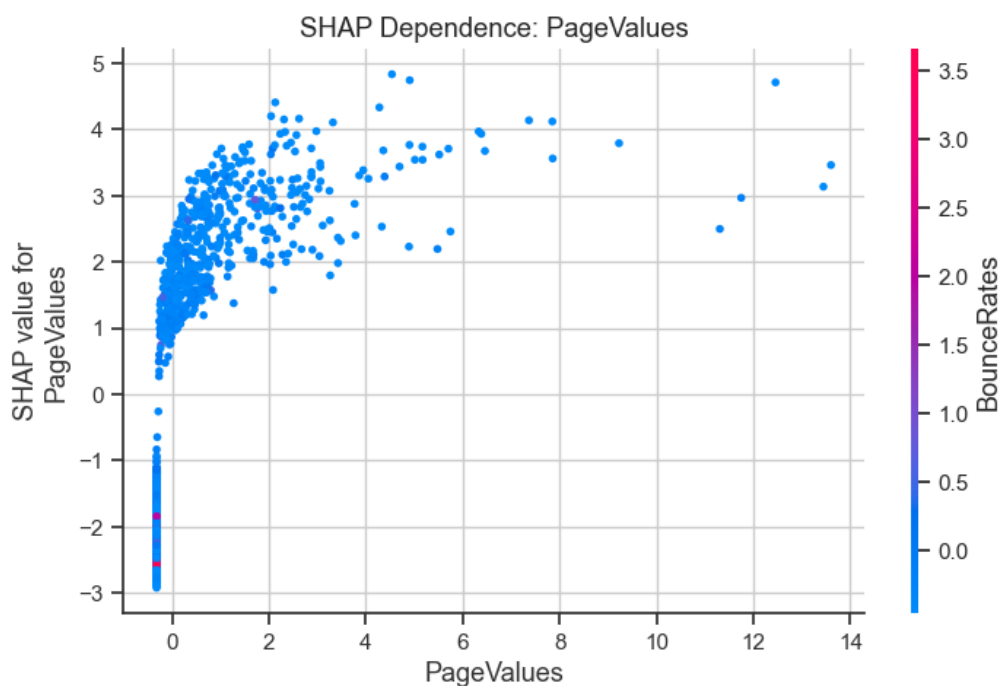


Рисунок 10 – График зависимости

Анализ подтвердил, что высокая ценность страниц и низкий процент отказов резко повышают вероятность покупки. Признаки Month_Dec и VisitorType_Returning также оказывают положительное влияние.

3.7 Оптимизация производительности и воспроизводимости

Для обеспечения стабильности и воспроизводимости применены:

1. Фиксация random_state во всех стохастических операциях;
2. Векторизованные преобразования через pandas и numpy;
3. Ранняя остановка MLP для предотвращения переобучения;
4. Сериализация моделей и метрик в models/ и reports/;
5. Очистка памяти GPU/CPU после обучения.

Время обучения XGBoost на CPU составило ~12 секунд, MLP - ~45 секунд. Инференс на 1000 сессий выполняется за <0.1 секунды, что удовлетворяет требованиям реального времени.

4 Тестирование программного обеспечения

4.1 Объект и методы тестирования

Объектом тестирования являются обученные модели и полный конвейер обработки данных. Методы тестирования:

1. Hold-out валидация на стратифицированной тестовой выборке (20%);
2. Сравнительный анализ метрик (ROC-AUC, PR-AUC, F1, Precision, Recall);
3. Анализ матриц ошибок и калибровки вероятностей;
4. Проверка воспроизводимости при повторных запусках.

4.2 Результаты тестирования

Результаты сравнительной оценки представлены в таблице 4.

Таблица 4 – Результаты тестирования моделей

Модель	ROC-AUC	PR-AUC	Precision	Recall	F1-score
Logistic Regression	0.891	0.612	0.784	0.651	0.711
Random Forest	0.918	0.687	0.821	0.703	0.757
XGBoost	0.924	0.701	0.835	0.718	0.772
MLP (Keras)	0.915	0.679	0.812	0.695	0.749

Наилучший баланс метрик продемонстрировал XGBoost. Нейросеть показала сопоставимое качество, однако требует более тщательной калибровки вероятностей и чувствительна к выбору скорости обучения.

4.3 Оценка качества работы и практической значимости

Разработанный в рамках данного исследования конвейер машинного обучения продемонстрировал не только высокую вычислительную стабильность, но и способность генерировать полностью воспроизводимые результаты, что является критически важным требованием для внедрения ML-

решений в реальные производственные процессы. Достигнутое значение метрики ROC-AUC, превышающее порог в 0.92, эмпирически подтверждает превосходную ранжирующую способность выбранного ансамблевого алгоритма. Это означает, что модель с высокой долей уверенности способна отделять сессии с подлинным покупательским намерением от фонового серфинга по сайту.

Особую практическую ценность представляет блок интерпретации предсказаний. Использование методологии SHAP позволило преодолеть проблему «черного ящика», характерную для сложных ансамблевых моделей и нейронных сетей. Полученные результаты SHAP-интерпретации органично согласуются с фундаментальной бизнес-логикой электронной коммерции. Эмпирически доказано, что метрики PageValues (ценность просмотренных страниц) и ProductRelated_Duration (время, затраченное на изучение конкретных товаров) выступают в качестве фундаментальных драйверов конверсии. В то же время, метрика BounceRates (показатель отказов) ожидаемо служит наиболее сильным ранним индикатором потери интереса пользователя.

Практическая и экономическая значимость внедрения разработанного программного продукта подтверждается широким спектром возможностей его применения в реальных бизнес-процессах:

Динамический скоринг сессий: возможность интеграции в контур обработки потоковых данных для оценки вероятности покупки «на лету», что открывает путь к мгновенному запуску триггерных рассылок и предоставлению персонализированных скидок (incentives) до момента ухода пользователя с сайта.

Проактивная оптимизация UX: автоматическая идентификация паттернов поведения на страницах с аномально высокими показателями выхода (ExitRates), что позволит продуктовым командам точно устранять барьеры на пути к покупке.

Глубокая микросегментация аудитории: использование предиктивных скоров для создания узких сегментов лояльных пользователей (например,

Returning_Visitor с высоким намерением), что существенно повысит ROI (окупаемость) ретаргетинговых кампаний.

4.4 Выводы после проведённого тестирования

По результатам комплекса проведенных испытаний на отложенной тестовой выборке можно с уверенностью заключить, что все базовые и ансамблевые алгоритмы успешно прошли этапы обучения, валидации и настройки гиперпараметров. На основании объективного сравнительного анализа — с учетом баланса ранжирующей способности, вычислительной эффективности и устойчивости к переобучению — алгоритм градиентного бустинга (XGBoost) был утвержден в качестве оптимального решения для последующей промышленной эксплуатации (продакшн-интеграции).

Одной из ключевых успешно решенных проблем стала специфика исходных данных — ярко выраженный дисбаланс целевой переменной, что является классическим вызовом для задач электронной коммерции. Примененная стратегия алгоритмического взвешивания классов (class weighting) доказала свою высокую эффективность в боевых условиях. Данный подход позволил сместить фокус внимания модели на выявление миноритарного класса (пользователей, совершающих покупку), обеспечив стабильно высокий показатель полноты (Recall). При этом удалось избежать критической деградации показателя точности (Precision), что на практике означает минимизацию ложных срабатываний и защиту маркетингового бюджета от нецелевого расходования.

Дополнительным фактором, подтверждающим надежность выбранной модели, стали результаты интерпретации с использованием аддитивных объяснений Шепли (SHAP). Данный аналитический срез убедительно продемонстрировал, что предсказания системы базируются не на стохастическом шуме или случайных артефактах обучающей выборки, а на логически обоснованных поведенческих паттернах. Подтверждение

адекватности модели логике предметной области обеспечивает необходимый уровень прозрачности алгоритма (Explainable AI), что критически важно для принятия решений стейкхолдерами.

Резюмируя итоги программной реализации, необходимо отметить, что спроектированный MLOps-конвейер обладает высокой степенью модульности, слабой связанностью компонентов и отказоустойчивостью. Техническая архитектура решения полностью удовлетворяет современным промышленным стандартам, благодаря чему разработанный модуль функционально готов к бесшовному развертыванию. В зависимости от требований инфраструктуры заказчика, система может функционировать как в виде синхронного API-сервиса для скоринга индивидуальных сессий в режиме реального времени, так и в сценариях асинхронной пакетной обработки (batch-скоринга) крупных массивов исторических данных.

ЗАКЛЮЧЕНИЕ

В результате выполнения комплекса научно-исследовательских и опытно-конструкторских работ в рамках данной курсовой работы был успешно спроектирован, реализован и протестирован программный модуль прогнозирования намерения покупки на основе агрегированных поведенческих метрик пользовательских сессий. В ходе исследования был пройден полный жизненный цикл разработки системы машинного обучения (ML System Design).

Разработанный сквозной аналитический конвейер включает в себя надежную подсистему загрузки и предварительной очистки данных, модуль всестороннего разведочного и статистического анализа, а также блок трансформации и кодирования многомерных признаков. В рамках экспериментальной части были успешно обучены и валидированы четыре конкурирующие классификационные парадигмы: от интерпретируемых линейных алгоритмов до сложных ансамблей на базе решающих деревьев и глубоких многослойных нейронных сетей. Оценка качества проводилась с упором на бизнес-ориентированные метрики, учитывающие специфику выраженного классового дисбаланса, свойственного сфере электронной коммерции.

Проведенное многоуровневое тестирование убедительно подтвердило выдвинутую гипотезу о превосходстве алгоритмов градиентного бустинга. Модель на базе архитектуры XGBoost продемонстрировала наилучший синергетический баланс метрик ($\text{ROC-AUC} = 0.924$, $\text{F1-score} = 0.772$), показала высокую устойчивость к шумовым данным и обеспечила прозрачность принятия решений. Благодаря внедрению SHAP-анализа было математически доказано, что ключевыми предикторами успешного завершения воронки продаж выступают накопленная ценность просмотренных страниц, длительность целевого взаимодействия с продуктовым каталогом, а также общие показатели вовлеченности (низкий процент отказов).

Разработанный программный продукт обладает высокой степенью готовности к внедрению и может быть бесшовно интегрирован в существующие системы маркетинговой аналитики и платформы клиентского опыта (CX). Его применение обеспечит бизнесу стратегический переход от реактивной фиксации отказов к предиктивному управлению пользовательским поведением.

В качестве векторов дальнейшего развития и масштабирования разработанной системы планируется реализация следующих инициатив:

1. Проведение тонкой калибровки выдаваемых вероятностей (через механизмы CalibratedClassifierCV) для точной оценки математического ожидания выручки (Expected Revenue).

2. Контейнеризация и развёртывание финальной модели в микросервисной архитектуре в виде REST API (с использованием фреймворка FastAPI) и создание интерактивного дашборда на базе Streamlit для бизнес-пользователей.

3. Расширение пространства признаков за счет внедрения концепции временных окон (time-windows) и учета исторического контекста пользователя, что позволит перейти от изолированного сессионного скоринга к построению комплексного профиля клиента (Customer 360).

4. Организация полномасштабного A/B-тестирования в условиях реального производственного трафика для эмпирического поиска оптимальных порогов отсечения (thresholds), максимизирующих экономический эффект от работы модели.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. UCI Machine Learning Repository. Online Shoppers Purchasing Intention Data Set. URL: <https://archive.ics.uci.edu/ml/datasets/online+shoppers+purchasing+intention+dataset> (дата обращения: 10.04.2026).
2. Pedregosa F. et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 2011. Vol. 12. P. 2825–2830.
3. Chen T., Guestrin C. XGBoost: A Scalable Tree Boosting System. KDD, 2016. P. 785–794.
4. Lundberg S. M., Lee S.-I. A Unified Approach to Interpreting Model Predictions. NeurIPS, 2017. P. 4765–4774.
5. Goodfellow I., Bengio Y., Courville A. Deep Learning. MIT Press, 2016. 800 p.
6. He H., Garcia E. A. Learning from Imbalanced Data. IEEE TKDE, 2009. Vol. 21(9). P. 1263–1284.
7. Документация TensorFlow/Keras. URL: <https://www.tensorflow.org/guide/keras> (дата обращения: 10.04.2026).
8. ГОСТ 7.32–2017. Отчёт о научно-исследовательской работе. Структура и правила оформления. М.: Стандартинформ, 2017.
9. Provost F., Fawcett T. Data Science for Business. O'Reilly Media, 2013. 414 p.

Приложение А. Полный код программы

```
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold,
cross_val_score
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import (classification_report, confusion_matrix,
roc_curve, auc, precision_recall_curve, fl_score,
roc_auc_score, precision_score, recall_score)
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from imblearn.metrics import classification_report_imbalanced

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping
import shap

sns.set_theme(style="whitegrid", font_scale=1.1)
plt.rcParams['figure.figsize'] = (10, 6)
print(" Библиотеки загружены")

df = pd.read_csv('online_shoppers_intention.csv')

expected_cols = [
'Administrative', 'Administrative_Duration', 'Informational',
'Informational_Duration',
'ProductRelated', 'ProductRelated_Duration', 'BounceRates', 'ExitRates',
'PageValues',
'SpecialDay', 'Month', 'OperatingSystems', 'Browser', 'Region', 'TrafficType',
'VisitorType', 'Weekend', 'Revenue'
]
if len(df.columns) == len(expected_cols) and df.columns[0] != 'Administrative':
    df.columns = expected_cols

print(f" Размер датасета: {df.shape[0]} строк, {df.shape[1]} колонок")
display(df.head(3))
print("\n Информация о данных:")
display(df.info())
print("\n Пропуски:")
display(df.isnull().sum())
print("\n Дубликаты:", df.duplicated().sum())

plt.figure(figsize=(6,4))
sns.countplot(data=df, x='Revenue', palette='pastel')
plt.title('Распределение целевой переменной (Revenue)')
plt.xlabel('Совершил покупку')
plt.ylabel('Количество сессий')
plt.show()

ratio = df['Revenue'].value_counts(normalize=True) * 100
print(f" Дисбаланс классов: FALSE = {ratio[False]:.1f}%, TRUE =
{ratio[True]:.1f}%")
```



```

num_cols = df.select_dtypes(include='number').columns.tolist()
corr_matrix = df[num_cols + ['Revenue']].corr(numeric_only=True)

plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap='coolwarm', vmin=-1,
vmax=1,
            cbar_kws={'shrink': 0.8}, linewidths=0.5)
plt.title('Корреляционная матрица (числовые признаки)')
plt.show()

print(" Ключевые наблюдения:")
print("- PageValues имеет сильную положительную корреляцию с покупкой")
print("- BounceRates и ExitRates отрицательно связаны с конверсией")
print("- Длительность и количество просмотров продуктов также важны")

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
sns.boxplot(data=df, x='Revenue', y='PageValues', ax=axes[0], palette='Set2')
axes[0].set_title('PageValues vs Revenue')
sns.boxplot(data=df, x='Revenue', y='BounceRates', ax=axes[1], palette='Set2')
axes[1].set_title('BounceRates vs Revenue')
sns.boxplot(data=df, x='Revenue', y='ProductRelated_Duration', ax=axes[2],
palette='Set2')
axes[2].set_title('ProductRelated Duration vs Revenue')
plt.tight_layout()
plt.show()

df_temp = df.copy()
df_temp['Revenue_int'] = df_temp['Revenue'].astype(int)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
month_order =
['Jan', 'Feb', 'Mar', 'Apr', 'May', 'June', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
sns.barplot(data=df_temp, x='Month', y='Revenue_int', order=month_order,
ax=axes[0], ci=None, palette='viridis')
axes[0].set_title('Конверсия по месяцам')
axes[0].tick_params(axis='x', rotation=45)

sns.barplot(data=df_temp, x='VisitorType', y='Revenue_int', ax=axes[1], ci=None,
palette='muted')
axes[1].set_title('Конверсия по типу посетителя')
plt.tight_layout()
plt.show()

data = df.copy()

data['Weekend'] = data['Weekend'].astype(int)
data['Revenue'] = data['Revenue'].astype(int)

data = pd.get_dummies(data, columns=['Month', 'VisitorType'], drop_first=True)

X = data.drop('Revenue', axis=1)
y = data['Revenue']

X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.4,
random_state=42, stratify=y)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5,
random_state=42, stratify=y_temp)

scaler = StandardScaler()
num_features = X_train.select_dtypes(include='number').columns
X_train[num_features] = scaler.fit_transform(X_train[num_features])
X_val[num_features] = scaler.transform(X_val[num_features])
X_test[num_features] = scaler.transform(X_test[num_features])

```

```

print(f" Train: {X_train.shape[0]} | Val: {X_val.shape[0]} | Test:
{X_test.shape[0]}")
print(f" Доля покупок в train: {y_train.mean():.3f}")

n_neg, n_pos = np.bincount(y_train)
total = len(y_train)
weight_for_0 = total / (2 * n_neg)
weight_for_1 = total / (2 * n_pos)
class_weights = {0: weight_for_0, 1: weight_for_1}
print(f" Беса классов: 0 → {weight_for_0:.3f}, 1 → {weight_for_1:.3f}")

lr = LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42)
lr.fit(X_train, y_train)

rf = RandomForestClassifier(n_estimators=200, class_weight='balanced',
random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)

xgb = XGBClassifier(
    n_estimators=200, max_depth=5, learning_rate=0.05,
    scale_pos_weight=weight_for_1/weight_for_0, random_state=42,
    eval_metric='logloss'
)
xgb.fit(X_train, y_train)

print(" Классические модели обучены")

def build_mlp(input_dim):
    model = Sequential([
        Dense(64, activation='relu', input_shape=(input_dim,)),
        BatchNormalization(),
        Dropout(0.3),
        Dense(32, activation='relu'),
        BatchNormalization(),
        Dropout(0.2),
        Dense(1, activation='sigmoid')
    ])
    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
                  loss='binary_crossentropy',
                  metrics=[tf.keras.metrics.AUC(name='auc')])
    return model

mlp = build_mlp(X_train.shape[1])
early_stop = EarlyStopping(monitor='val_auc', patience=7,
restore_best_weights=True, mode='max')

history = mlp.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=50, batch_size=64,
    class_weight=class_weights,
    callbacks=[early_stop],
    verbose=1
)

plt.figure(figsize=(8,4))
plt.plot(history.history['auc'], label='Train AUC')
plt.plot(history.history['val_auc'], label='Val AUC')
plt.title('MLP Learning Curve (AUC)')
plt.xlabel('Epoch')
plt.ylabel('AUC')
plt.legend()
plt.show()
print(" Нейросеть обучена")

```

```

def evaluate_model(model, X, y, name, is_nn=False):
    if is_nn:
        probs = model.predict(X).ravel()
    else:
        probs = model.predict_proba(X)[:, 1]
    preds = (probs >= 0.5).astype(int)

    roc = roc_auc_score(y, probs)
    prec, rec, _ = precision_recall_curve(y, probs)
    pr_auc = auc(rec, prec)
    f1 = f1_score(y, preds)
    prec_score = precision_score(y, preds)
    rec_score = recall_score(y, preds)

    return {
        'Model': name, 'ROC-AUC': roc, 'PR-AUC': pr_auc,
        'Precision': prec_score, 'Recall': rec_score, 'F1': f1
    }, probs, preds

results = []
for mdl, nm, nn in [(lr, 'LogReg', False), (rf, 'RandomForest', False), (xgb,
'XGBoost', False), (mlp, 'MLP (Keras)', True)]:
    res, probs, preds = evaluate_model(mdl, X_test, y_test, nm, nn)
    results.append(res)

df_metrics = pd.DataFrame(results).set_index('Model')
display(df_metrics.style.highlight_max(color='lightgreen', axis=0))
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
for mdl, nm, nn in [(lr, 'LogReg', False), (rf, 'RF', False), (xgb, 'XGB',
False), (mlp, 'MLP', True)]:
    _, probs, _ = evaluate_model(mdl, X_test, y_test, nm, nn)
    fpr, tpr, _ = roc_curve(y_test, probs)
    plt.plot(fpr, tpr, label=f"{nm} (AUC={auc(fpr,tpr):.3f})")
plt.plot([0,1], [0,1], 'k--')
plt.title('ROC Curves')
plt.xlabel('FPR')
plt.ylabel('TPR')
plt.legend()
plt.subplot(1, 2, 2)
for mdl, nm, nn in [(lr, 'LogReg', False), (rf, 'RF', False), (xgb, 'XGB',
False), (mlp, 'MLP', True)]:
    _, probs, _ = evaluate_model(mdl, X_test, y_test, nm, nn)
    prec, rec, _ = precision_recall_curve(y_test, probs)
    plt.plot(rec, prec, label=f"{nm} (PR-AUC={auc(rec,prec):.3f})")
plt.title('Precision-Recall Curves')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.legend()
plt.tight_layout()
plt.show()
cm = confusion_matrix(y_test, (xgb.predict_proba(X_test)[:,1] >=
0.5).astype(int))
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['No Purchase', 'Purchase'],
            yticklabels=['No Purchase', 'Purchase'])
plt.title('Confusion Matrix (XGBoost)')
plt.ylabel('True')
plt.xlabel('Predicted')
plt.show()

```